Database Management – Unit Notes

GTU Diploma Engineering | Information Technology | Course Code: DI03016041

Format: Simple English, beginner-friendly, exam-oriented. Each topic includes definition, 3 uses, 3 features, applications, and a real-life example.

UNIT 1 – Introduction to Database Systems

1.1 Data and Information

Definition: Data is a collection of raw facts such as numbers, names, or dates. Information is processed data that has a useful meaning.

3 Uses:

- Stores facts for later processing.
- Provides input for database operations.
- Helps produce meaningful results.

3 Features:

- Data may be raw.
- Information is processed and meaningful.
- Both can be stored and handled by computer systems.

Applications: Used in schools, banks, hospitals, offices, and business systems.

Real-Life Example: A student's marks are data; calculating the total and percentage from those marks produces useful information.

1.1.2 Database and Database Management System (DBMS)

Definition: A database is an organized collection of related data. A Database Management System (DBMS) is software used to create, store, retrieve, update, and manage data in databases.

3 Uses:

- Stores organized data.
- Retrieves required records.
- Allows data to be inserted, updated, and deleted.

3 Features:

- Provides controlled data management.
- Supports database operations.
- Allows users and applications to work with stored data.

Applications: Used in banking, education, hospitals, shopping, and reservation systems.

Real-Life Example: A college DBMS stores student details, subjects, marks, and course information and allows authorized users to retrieve them.

1.1.3 Purpose of Database System and Application

Definition: The purpose of a database system is to store related data in an organized form and provide convenient access to that data.

3 Uses:

- Stores large amounts of related data.
- Makes data retrieval easier.

- Supports applications that need shared data.

3 Features:

- Organized storage.
- Supports data retrieval.
- Can be used by different database applications.

Applications: Used in banking, railway reservation, hospitals, libraries, colleges, and online shopping.

Real-Life Example: A railway reservation system stores train, passenger, seat, and booking information in a database.

1.1.4 Metadata

Definition: Metadata is data that describes other data. It gives information about the structure, meaning, type, or properties of stored data.

3 Uses:

- Describes database objects.
- Helps users understand stored data.
- Supports database management.

3 Features:

- Describes data rather than the actual business record.
- Can contain information about tables and columns.
- Helps identify data structure.

Applications: Used in databases, data catalogs, files, and information systems.

Real-Life Example: In a student table, metadata can describe that Student_ID is an integer column and Name is a character column.

1.1.5 Data Items, Fields and Records

Definition: A data item is a single value. A field is a named place or attribute used to store a value, and a record is a collection of related fields describing one object.

3 Uses:

- Organizes information into meaningful parts.
- Helps represent database records.
- Makes data easier to store and retrieve.

3 Features:

- Fields have names.
- Records contain related fields.
- Data items represent individual values.

Applications: Used in relational tables and database applications.

Real-Life Example: In a student table, Roll_No, Name, and Course are fields; the values for one student together form one record.

1.1.6 Data Dictionary

Definition: A data dictionary is a collection of information about the data stored in a database. It describes database objects and their properties.

3 Uses:

- Provides information about database structure.
- Helps understand tables and fields.

- Supports database administration.

3 Features:

- Contains descriptions of data.
- Contains structural information.
- Helps database users and administrators.

Applications: Used by database administrators and database systems.

Real-Life Example: A data dictionary can show a table's column names, data types, and constraints.

1.2 File-Oriented System versus Database System

Definition: A file-oriented system stores data in separate application files, while a database system stores related data in a managed database.

3 Uses:

- Helps understand two methods of data storage.
- Provides a basis for choosing a storage approach.
- Helps identify how database systems organize shared data.

3 Features:

- File systems use separate files.
- Database systems use a DBMS.
- Database systems provide managed access to shared data.

Applications: Used when studying database architecture and selecting data-management methods.

Real-Life Example: A college may have separate files for student and fee data in a file system, while a DBMS can manage related student and fee tables together.

1.3 Database Administrator (DBA)

Definition: A Database Administrator (DBA) is the person responsible for managing and controlling a database system.

3 Uses:

- Manages database structure and operation.
- Controls database access.
- Helps maintain database availability and security.

3 Features:

- Works with database software.
- Manages users and permissions.
- Monitors and maintains databases.

Applications: Used in banks, companies, hospitals, colleges, and other organizations with databases.

Real-Life Example: A bank DBA manages access to customer account databases and helps maintain their reliable operation.

1.4 Schema, Sub-Schema and Instances

Definition: A schema describes the overall structure of a database. A sub-schema describes the part of the database needed by a particular user or application. An instance is the actual data stored at a particular time.

3 Uses:

- Describes database organization.
- Supports different user views.
- Helps understand database structure and current data.

3 Features:

- Schema is the overall design.
- Sub-schema represents a selected view.
- Instance changes as data changes.

Applications: Used in database design, administration, and application development.

Real-Life Example: A college database schema may contain Student and Course tables, while a teacher's sub-schema may show only information needed for teaching.

1.5 Data Abstraction

Definition: Data abstraction means hiding unnecessary details of data storage and showing users only the information they need.

3 Uses:

- Makes database use easier.
- Hides complex storage details.
- Provides different levels of data view.

3 Features:

- Uses levels of abstraction.
- Hides lower-level details.
- Provides suitable views for different users.

Applications: Used in DBMS design and user interfaces.

Real-Life Example: A student sees marks and attendance, while the database system internally manages how those values are stored.

1.5.1 Internal Level

Definition: The internal level is the lowest level of data abstraction. It describes how data is physically stored in the database system.

3 Uses:

- Describes physical data storage.
- Helps manage storage structures.
- Supports efficient database implementation.

3 Features:

- Closest to physical storage.
- Contains storage details.
- Hidden from normal users.

Applications: Used by database system designers and administrators.

Real-Life Example: A DBMS may store table data using particular files, blocks, or indexes internally.

1.5.2 Conceptual Level

Definition: The conceptual level describes the overall logical structure of the complete database, without focusing on physical storage details.

3 Uses:

- Describes database-wide structure.
- Shows relationships among data.
- Supports database design.

3 Features:

- Middle level of abstraction.
- Independent of physical storage details.
- Represents the logical database structure.

Applications: Used by database designers and database administrators.

Real-Life Example: A conceptual design can show Student, Course, and Enrollment entities and their relationships.

1.5.3 External Level

Definition: The external level is the highest level of data abstraction. It shows only the part of the database required by a particular user or application.

3 Uses:

- Provides user-specific views.
- Hides unnecessary data.
- Simplifies database use.

3 Features:

- Closest to users.
- Different users can have different views.
- Hides internal details.

Applications: Used in applications where different users need different information.

Real-Life Example: A student may see personal marks, while an administrator can see additional academic records.

1.6 Data Independence

Definition: Data independence means that changes at one level of the database system do not require unnecessary changes at higher levels.

3 Uses:

- Makes database changes easier.
- Reduces application changes.
- Supports flexible database design.

3 Features:

- Separates data levels.
- Reduces dependency between levels.
- Supports database maintenance.

Applications: Used in DBMS architecture and database maintenance.

Real-Life Example: Changing the internal storage method should not require changing a student's screen that displays their marks.

UNIT 2 – Entity Relationship Model

2.1 Entity

Definition: An entity is a real-world object or concept about which data is stored in a database.

3 Uses:

- Identifies objects in a system.
- Provides data for database design.
- Forms the basis of ER diagrams.

3 Features:

- Has identifiable properties.
- Can have attributes.
- Can participate in relationships.

Applications: Used in ER modeling for systems such as colleges, banks, hospitals, and shops.

Real-Life Example: In a college database, Student is an entity because information about students is stored.

2.1.2 Attributes

Definition: An attribute is a property or characteristic that describes an entity.

3 Uses:

- Describes entities.
- Provides details stored in a database.
- Helps identify required data fields.

3 Features:

- Belongs to an entity.
- Stores a particular property.
- Can have a specific data value.

Applications: Used in ER diagrams and database table design.

Real-Life Example: For a Student entity, Roll_No, Name, and Course can be attributes.

2.1.3 Relationship

Definition: A relationship shows an association between two or more entities.

3 Uses:

- Connects related entities.
- Represents real-world associations.
- Helps design database relationships.

3 Features:

- Connects entities.
- Can have different degrees.
- Can have participation rules.

Applications: Used in ER models for college, banking, hospital, and business databases.

Real-Life Example: Student and Course can be related through an Enrollment relationship.

2.1.3.1 Participation

Definition: Participation describes whether all or only some entities of an entity set must take part in a relationship.

3 Uses:

- Defines relationship participation.
- Helps represent business rules.
- Supports accurate ER modeling.

3 Features:

- Can involve mandatory participation.
- Can involve partial participation.
- Describes entity involvement in relationships.

Applications: Used when designing ER diagrams with real-world participation rules.

Real-Life Example: If every employee must belong to a department, Employee has mandatory participation in that relationship.

2.1.3.2 Recursive Relationship

Definition: A recursive relationship is a relationship in which an entity is related to another entity of the same entity set.

3 Uses:

- Represents self-related data.
- Models hierarchical relationships.
- Helps represent relationships within one entity type.

3 Features:

- Uses the same entity set.
- Can have different roles.
- Represents self-reference.

Applications: Used in employee hierarchies, organizational structures, and reporting systems.

Real-Life Example: An Employee entity can have a relationship called Reports_To where one employee reports to another employee.

2.1.3.3 Degree of Relationship Set

Definition: The degree of a relationship set is the number of entity sets participating in that relationship.

3 Uses:

- Identifies participating entity sets.
- Helps classify relationships.
- Supports ER diagram design.

3 Features:

- Unary uses one entity set.
- Binary uses two entity sets.
- Ternary uses three entity sets.

Applications: Used when designing ER models.

Real-Life Example: Student–Course is a binary relationship because two entity sets participate.

2.2 Mapping Cardinality

Definition: Mapping cardinality specifies the number of entities that can be associated with entities of another set in a relationship.

3 Uses:

- Defines relationship quantity.
- Helps design correct database relationships.
- Supports conversion of ER diagrams to tables.

3 Features:

- Includes one-to-one relationships.
- Includes one-to-many relationships.
- Includes many-to-many relationships.

Applications: Used in ER diagrams and relational database design.

Real-Life Example: In a college, one department may have many students, representing a one-to-many relationship.

2.3 ER Diagrams

Definition: An ER diagram is a graphical representation of entities, attributes, and relationships in a database.

3 Uses:

- Helps visualize database design.
- Shows relationships between entities.
- Supports conversion into relational tables.

3 Features:

- Uses graphical symbols.
- Represents entities and attributes.
- Shows relationships.

Applications: Used during database analysis and design.

Real-Life Example: A college ER diagram can show Student, Course, and Enrollment and the relationships between them.

2.4 Weak Entity Sets

Definition: A weak entity set is an entity set that cannot be uniquely identified by its own attributes alone and depends on another entity set.

3 Uses:

- Represents dependent objects.
- Helps model entities without complete independent identification.
- Supports detailed ER modeling.

3 Features:

- Depends on another entity.
- Has a partial identifying key.
- Uses an identifying relationship.

Applications: Used in systems where one record depends on another.

Real-Life Example: An Employee_Dependent entity may depend on Employee for its identification.

2.5 Enhanced ER Model

Definition: The Enhanced ER model extends the basic ER model with concepts such as superclass, subclass, specialization, generalization, and aggregation.

3 Uses:

- Models complex real-world systems.

- Provides more detailed database design.
- Represents inheritance-like relationships.

3 Features:

- Includes superclass and subclass.
- Supports specialization and generalization.
- Supports aggregation.

Applications: Used for complex database requirements.

Real-Life Example: An Employee superclass can have subclasses such as Manager and Clerk.

2.5.1 Subclass and Superclass

Definition: A superclass is a general entity type, while a subclass is a specialized entity type that inherits common properties from the superclass.

3 Uses:

- Organizes related entities.
- Reduces repeated definitions.
- Represents specialized entity types.

3 Features:

- Superclass contains common properties.
- Subclass represents a specialized group.
- Supports inheritance of attributes.

Applications: Used in enhanced ER modeling.

Real-Life Example: Employee can be a superclass, while Manager can be a subclass with additional manager-specific information.

2.5.2 Generalization

Definition: Generalization is the process of combining similar lower-level entity types into a higher-level superclass.

3 Uses:

- Combines common features.
- Reduces repeated information.
- Creates a general entity type.

3 Features:

- Works from specific to general.
- Groups similar entities.
- Produces a superclass.

Applications: Used in enhanced ER design.

Real-Life Example: Car and Truck can be generalized into a Vehicle superclass because both share common properties.

2.5.3 Specialization

Definition: Specialization is the process of dividing a higher-level entity into more specific subclasses.

3 Uses:

- Creates specialized entities.
- Represents different entity categories.
- Adds specific properties to subclasses.

3 Features:

- Works from general to specific.
- Creates subclasses.
- Allows specialized attributes.

Applications: Used in enhanced ER modeling.

Real-Life Example: Employee can be specialized into Manager and Technician.

2.5.4 Aggregation

Definition: Aggregation is an abstraction in which a relationship and its participating entities are treated as a higher-level entity for another relationship.

3 Uses:

- Models complex relationships.
- Simplifies complex ER designs.
- Allows relationships to participate in higher-level relationships.

3 Features:

- Treats a relationship as a higher-level unit.
- Used in complex ER models.
- Connects relationship-level concepts.

Applications: Used in enhanced ER diagrams.

Real-Life Example: A Works_On relationship between Employee and Project can participate in another relationship such as Monitored_By.

2.6 Converting ER Diagrams to Database

Definition: Converting an ER diagram to a database means transforming entities, attributes, and relationships into relational tables and keys.

3 Uses:

- Creates tables from entities.
- Creates keys for identification.
- Represents relationships in relational form.

3 Features:

- Entities become tables.
- Attributes become columns.
- Relationships are represented using keys or relationship tables.

Applications: Used during database implementation after ER modeling.

Real-Life Example: A Student entity becomes a Student table, and a many-to-many Student–Course relationship can become an Enrollment table.

UNIT 3 – Structured Query Language

3.1 SQL Introduction and Basic Commands

Definition: SQL (Structured Query Language) is a language used to create, manage, manipulate, and retrieve data in relational databases.

3 Uses:

- Creates database objects.
- Manipulates stored data.
- Retrieves required records.

3 Features:

- Uses commands for database operations.
- Works with relational data.
- Supports different categories of commands.

Applications: Used in database applications such as banking, college, hospital, and shopping systems.

Real-Life Example: A developer can use SQL to create a Student table and retrieve student records.

3.1.1 SQL Data Types

Definition: SQL data types specify the kind of value that a database column can store.

3 Uses:

- Defines column storage type.
- Helps store suitable values.
- Supports correct database design.

3 Features:

- Different types store different values.
- Applied to columns.
- Helps control stored data format.

Applications: Used while creating and modifying database tables.

Real-Life Example: A Student_ID may use an integer type, while Student_Name may use a character type.

3.1.2 DDL Commands

Definition: Data Definition Language (DDL) commands define and modify database structures. The syllabus includes CREATE, ALTER, TRUNCATE, and DROP.

3 Uses:

- Creates database objects.
- Changes table structure.
- Removes or clears database structures/data as specified by the command.

3 Features:

- CREATE creates objects.
- ALTER changes structure.
- TRUNCATE and DROP perform removal-related operations.

Applications: Used by database developers and administrators.

Real-Life Example: CREATE TABLE can create a Student table, while ALTER TABLE can add a new column.

3.1.3 DML Commands

Definition: Data Manipulation Language (DML) commands are used to add, modify, and remove records in database tables.

3 Uses:

- Inserts records.
- Updates existing records.
- Deletes records.

3 Features:

- INSERT adds data.
- UPDATE changes data.
- DELETE removes selected records.

Applications: Used in database applications whenever stored records need to be changed.

Real-Life Example: A college application can use INSERT to add a new student's record.

3.1.4 Data Query Language – SELECT

Definition: SELECT is used to retrieve data from one or more database tables.

3 Uses:

- Retrieves records.
- Selects required columns.
- Can retrieve records according to conditions.

3 Features:

- Reads data.
- Can use conditions.
- Can retrieve selected columns or rows.

Applications: Used in reports, search screens, dashboards, and database applications.

Real-Life Example: SELECT can display the names and marks of students from a Student table.

3.1.5 GRANT and REVOKE

Definition: GRANT gives database privileges to a user, while REVOKE removes previously given privileges.

3 Uses:

- Controls database access.
- Gives required permissions.
- Removes permissions when necessary.

3 Features:

- Works with user privileges.
- Supports access control.
- GRANT and REVOKE perform opposite permission operations.

Applications: Used in database security and user management.

Real-Life Example: A database administrator can grant a user permission to read a table and later revoke that permission.

3.1.6 Other SQL Commands

Definition: SQL provides commands and clauses such as DESCRIBE, DISTINCT, ORDER BY, GROUP BY, and HAVING for examining and organizing query results.

3 Uses:

- Shows table structure.
- Removes duplicate values from query results.
- Sorts and groups query results.

3 Features:

- DESCRIBE shows structure.
- ORDER BY sorts results.
- GROUP BY and HAVING support grouped results.

Applications: Used for database queries, reports, and analysis.

Real-Life Example: ORDER BY can display student names in alphabetical order.

3.2 SQL Views

Definition: A SQL view is a virtual table based on the result of a query. It provides a selected way to access data.

3 Uses:

- Provides a specific data view.
- Simplifies repeated queries.
- Can show selected columns or rows.

3 Features:

- Based on a query.
- Acts like a virtual table.
- Can present selected database information.

Applications: Used in reporting and applications where users need a particular view of database data.

Real-Life Example: A college can create a view that shows only student names and marks.

3.3 SQL Functions

Definition: SQL functions are built-in operations used to perform calculations or process values in SQL queries.

3 Uses:

- Perform calculations.
- Process data values.
- Help produce useful query results.

3 Features:

- Can operate on data values.
- Can be used inside queries.
- Support data processing.

Applications: Used in reports, calculations, and data analysis.

Real-Life Example: An aggregate function can calculate the average marks of students.

3.4 SQL Operators

Definition: SQL operators are symbols or keywords used to compare, calculate, or combine values in SQL expressions.

3 Uses:

- Compare values.
- Perform calculations.
- Build conditions in queries.

3 Features:

- Include comparison operations.
- Can be used in conditions.
- Help form SQL expressions.

Applications: Used in SELECT, UPDATE, DELETE, and other SQL statements.

Real-Life Example: A query can use an operator to find students whose marks are greater than 50.

3.5 Set Operators

Definition: Set operators combine the results of two or more compatible SQL queries. The syllabus includes UNION, UNION ALL, INTERSECT, and MINUS.

3 Uses:

- Combines query results.
- Finds common results.
- Finds results present in one query but not another.

3 Features:

- UNION combines distinct results.
- UNION ALL keeps duplicates.
- INTERSECT finds common rows and MINUS finds rows in one result but not the other.

Applications: Used for comparing and combining query results.

Real-Life Example: Two queries can be combined with UNION to produce one list of customer account numbers.

3.6 Joins

Definition: A join combines related rows from two or more tables using a related column or condition.

3 Uses:

- Combines related table data.
- Retrieves information spread across tables.
- Supports multi-table queries.

3 Features:

- Works with multiple tables.
- Uses related data.
- Produces combined query results.

Applications: Used in relational databases where data is divided into related tables.

Real-Life Example: A query can join Student and Course-related tables to display student enrollment information.

3.7 SQL Constraints

Definition: SQL constraints are rules applied to table columns to control the type and validity of stored data.

3 Uses:

- Maintains valid data.
- Controls allowed values.
- Helps maintain database integrity.

3 Features:

- Applied to table columns.
- Enforces rules.

- Supports data integrity.

Applications: Used while creating and altering database tables.

Real-Life Example: A Primary Key constraint can ensure that each student record has a unique identifier.

3.7.2 Domain Integrity – NOT NULL and CHECK

Definition: Domain integrity ensures that column values follow defined rules. NOT NULL prevents a column from having NULL values, while CHECK restricts values according to a condition.

3 Uses:

- Controls valid column values.
- Prevents missing required values.
- Applies conditions to stored data.

3 Features:

- NOT NULL requires a value.
- CHECK tests a condition.
- Both are column-level data rules.

Applications: Used in table design to maintain valid data.

Real-Life Example: A student's name can be defined as NOT NULL, while marks can use CHECK to restrict values to an acceptable range.

3.7.3 Entity Integrity – UNIQUE and PRIMARY KEY

Definition: Entity integrity helps identify records correctly. UNIQUE prevents duplicate values in a constrained column, while PRIMARY KEY uniquely identifies each record.

3 Uses:

- Identifies records.
- Prevents unwanted duplicate values.
- Supports reliable table design.

3 Features:

- PRIMARY KEY identifies records.
- UNIQUE prevents duplicate constrained values.
- Keys are used for row identification.

Applications: Used in relational database table design.

Real-Life Example: Student_ID can be a primary key so that each student has a unique identifier.

3.7.4 Referential Integrity – FOREIGN KEY and REFERENCES

Definition: Referential integrity keeps relationships between tables valid. A foreign key refers to a key in another table, commonly using a REFERENCES definition.

3 Uses:

- Maintains valid table relationships.
- Connects related tables.
- Prevents invalid references according to defined constraints.

3 Features:

- Uses foreign keys.
- Links parent and child tables.
- Depends on referenced key values.

Applications: Used in relational databases with related tables.

Real-Life Example: An Enrollment table can use Student_ID as a foreign key referencing the Student table.

UNIT 4 – Normalization

4.1 Anomalies Created by Poor Database Design

Definition: Anomalies are unwanted problems that can occur when data is poorly organized in a database. Common problems include insertion, update, and deletion anomalies.

3 Uses:

- Helps identify poor database design.
- Shows why normalization is needed.
- Helps improve table organization.

3 Features:

- Can occur due to repeated data.
- Can cause inconsistent information.
- Can make database maintenance difficult.

Applications: Used during database design and normalization.

Real-Life Example: If the same customer address is stored in many rows, changing the address may require multiple updates and can create inconsistent data.

4.2 Normalization

Definition: Normalization is the process of organizing database tables to reduce unnecessary data repetition and improve data organization.

3 Uses:

- Reduces data redundancy.
- Improves database structure.
- Helps avoid data anomalies.

3 Features:

- Uses normal forms.
- Organizes related data.
- Uses dependencies to guide table design.

Applications: Used when designing relational databases.

Real-Life Example: A customer and order database can be divided into suitable tables so customer information is not unnecessarily repeated in every order.

4.2.2 Goals of Normalization

Definition: The main goals of normalization are to organize data properly, reduce unnecessary repetition, and reduce problems caused by poor database design.

3 Uses:

- Reduces redundant data.
- Helps prevent anomalies.
- Improves logical database organization.

3 Features:

- Focuses on dependencies.
- Divides data into suitable relations.
- Supports consistent database design.

Applications: Used during relational database design.

Real-Life Example: Separating Student and Department information can reduce repeated department details in many student records.

4.3.1 Prime vs Non-Prime Attributes

Definition: A prime attribute is an attribute that is part of a candidate key. A non-prime attribute is not part of any candidate key.

3 Uses:

- Helps analyze functional dependencies.
- Supports normalization.
- Helps identify key and non-key data.

3 Features:

- Prime attributes belong to candidate keys.
- Non-prime attributes do not.
- Used in normal-form analysis.

Applications: Used in normalization and relational schema analysis.

Real-Life Example: If Student_ID is part of a candidate key, it is a prime attribute; Student_Name may be non-prime.

4.3.2 Functional Dependency

Definition: A functional dependency exists when the value of one attribute or set of attributes determines the value of another attribute.

3 Uses:

- Helps identify relationships between attributes.
- Supports normalization.
- Helps determine suitable keys and table structures.

3 Features:

- Uses determinant and dependent attributes.
- Shows dependency between values.
- Important in normal-form analysis.

Applications: Used in database design and normalization.

Real-Life Example: If Student_ID determines Student_Name, then Student_ID $\rightarrow$ Student_Name represents a functional dependency.

4.3.2.1 Partial Functional Dependency

Definition: Partial functional dependency occurs when a non-prime attribute depends on only part of a composite candidate key rather than the whole key.

3 Uses:

- Helps identify 2NF violations.
- Supports decomposition of tables.
- Reduces dependency problems.

3 Features:

- Requires a composite key context.
- Involves a non-prime attribute.
- Depends on part of a key.

Applications: Used while converting a relation toward Second Normal Form.

Real-Life Example: If (Student_ID, Course_ID) is a composite key but Student_Name depends only on Student_ID, the dependency is partial.

4.3.2.2 Full Functional Dependency

Definition: Full functional dependency occurs when a non-prime attribute depends on the complete composite candidate key and not on only part of it.

3 Uses:

- Helps test Second Normal Form.
- Supports proper table design.
- Identifies complete dependencies.

3 Features:

- Uses the whole composite key.
- Does not depend on only part of the key.
- Important for 2NF.

Applications: Used in normalization analysis.

Real-Life Example: If Marks depends on both Student_ID and Course_ID together, the dependency is full.

4.3.2.3 Transitive Dependency

Definition: Transitive dependency occurs when a non-key attribute depends on another non-key attribute, which in turn depends on the key.

3 Uses:

- Helps identify 3NF violations.
- Supports database decomposition.
- Reduces indirect dependencies.

3 Features:

- Involves indirect dependency.
- Occurs through another attribute.
- Important for Third Normal Form.

Applications: Used during normalization to 3NF.

Real-Life Example: If Student_ID determines Dept_ID and Dept_ID determines Dept_Name, then Student_ID indirectly determines Dept_Name.

4.4.1 First Normal Form (1NF)

Definition: A relation is in First Normal Form when each field contains a single, atomic value and repeating groups are removed.

3 Uses:

- Removes repeating groups.
- Makes values atomic.
- Provides a basic normalized structure.

3 Features:

- Each cell contains one value.
- No repeating groups.
- Uses a tabular structure.

Applications: Used as the first step of normalization.

Real-Life Example: Instead of storing several phone numbers in one cell, separate related records can be used so each field contains one value.

4.4.2 Second Normal Form (2NF)

Definition: A relation is in Second Normal Form when it is in 1NF and every non-prime attribute is fully functionally dependent on the whole candidate key.

3 Uses:

- Removes partial dependencies.
- Improves table organization.
- Supports further normalization.

3 Features:

- Must first satisfy 1NF.
- Removes partial functional dependency.
- Important for composite keys.

Applications: Used in relational database normalization.

Real-Life Example: If Student_Name depends only on Student_ID in a table with key (Student_ID, Course_ID), Student_Name should be separated to remove the partial dependency.

4.4.3 Third Normal Form (3NF)

Definition: A relation is in Third Normal Form when it is in 2NF and has no transitive dependency of non-prime attributes on a candidate key.

3 Uses:

- Removes transitive dependencies.
- Improves data organization.
- Reduces repeated dependent information.

3 Features:

- Must satisfy 2NF.
- Removes transitive dependency.
- Separates indirectly dependent information.

Applications: Used to further improve relational database design.

Real-Life Example: Department details can be placed in a separate Department table when they depend on Dept_ID rather than directly on Student_ID.

4.4.4 Boyce-Codd Normal Form (BCNF)

Definition: A relation is in BCNF when, for every non-trivial functional dependency, the determinant is a candidate key.

3 Uses:

- Provides stronger dependency control.
- Helps improve relational design.
- Addresses certain dependency problems beyond 3NF.

3 Features:

- Uses functional dependencies.
- Requires determinants to be candidate keys.
- Is stronger than 3NF.

Applications: Used when a database design needs a stricter normal form.

Real-Life Example: If a non-trivial dependency has a determinant that is not a candidate key, the relation may need decomposition to satisfy BCNF.

UNIT 5 – Transaction Management

5.1 Transaction – Definition

Definition: A transaction is a sequence of database operations treated as one logical unit of work.

3 Uses:

- Performs a complete database task.
- Maintains correct database state.
- Allows a group of operations to be handled together.

3 Features:

- Contains one or more operations.
- Has a beginning and completion.
- Can be committed or rolled back.

Applications: Used in banking, shopping, reservation, and other database applications.

Real-Life Example: In a bank transfer, money is deducted from one account and added to another as one transaction.

5.1.2 Transaction State Transition Diagram

Definition: A transaction passes through different states during its execution, such as active, partially committed, committed, failed, and aborted.

3 Uses:

- Describes transaction progress.
- Helps understand transaction execution.
- Shows possible success and failure paths.

3 Features:

- Has multiple transaction states.
- Represents successful completion.
- Represents failure and rollback situations.

Applications: Used in transaction management and DBMS study.

Real-Life Example: During a bank transfer, a transaction may be active while operations execute and committed after all required operations finish successfully.

5.1.3 ACID Properties

Definition: ACID properties describe important characteristics that help database transactions remain reliable: Atomicity, Consistency, Isolation, and Durability.

3 Uses:

- Supports reliable transactions.
- Helps maintain correct database state.
- Supports safe concurrent database use.

3 Features:

- Atomicity treats a transaction as a unit.
- Consistency preserves valid database rules.
- Isolation separates concurrent transactions and Durability preserves committed results.

Applications: Used in banking, payment, booking, and other transaction-based systems.

Real-Life Example: In a money transfer, either the complete transfer is successfully applied or the transaction is rolled back.

5.1.4 Transaction Log

Definition: A transaction log is a record maintained by a DBMS containing information about transaction operations and changes.

3 Uses:

- Records transaction activity.
- Supports recovery after failures.
- Helps track database changes.

3 Features:

- Stores transaction-related information.
- Supports recovery.
- Used by the DBMS during transaction management.

Applications: Used in databases where recovery from system failure is important.

Real-Life Example: If a system crashes during a transaction, the transaction log can help the DBMS recover the database to a correct state.

5.2 Concurrency Control

Definition: Concurrency control is the process of managing simultaneous transactions so that database operations produce correct results.

3 Uses:

- Controls simultaneous database access.
- Helps maintain data consistency.
- Reduces problems caused by concurrent transactions.

3 Features:

- Handles multiple transactions.
- Uses control techniques.
- Maintains correct transaction behavior.

Applications: Used in banking, online shopping, reservation systems, and multi-user databases.

Real-Life Example: If two users try to reserve the last seat at the same time, concurrency control helps ensure that the database handles the operations correctly.

5.2.2 Problems of Concurrency Control

Definition: Concurrency problems occur when multiple transactions access or modify the same data at the same time without proper control.

3 Uses:

- Helps identify transaction conflicts.
- Supports selection of control methods.
- Helps maintain correct results.

3 Features:

- Can occur with simultaneous transactions.
- May cause incorrect or inconsistent results.
- Requires concurrency-control techniques.

Applications: Used in multi-user database systems.

Real-Life Example: If two transactions update the same account balance at the same time, uncontrolled execution may produce an incorrect final value.

5.2.3 Schedule and Its Types

Definition: A schedule is the order in which operations from multiple transactions are executed. Schedules can be classified according to how transaction operations are arranged.

3 Uses:

- Represents transaction execution order.
- Helps analyze concurrent transactions.
- Supports serializability analysis.

3 Features:

- Contains operations of transactions.
- Can be serial.
- Can be non-serial/concurrent.

Applications: Used in concurrency control and transaction analysis.

Real-Life Example: A schedule may execute all operations of Transaction T1 before T2, or interleave operations of T1 and T2.

5.3 Serializability of Transactions

Definition: Serializability is a property used to determine whether a concurrent schedule produces a result equivalent to some serial execution of transactions.

3 Uses:

- Checks correctness of concurrent schedules.
- Helps analyze transaction ordering.
- Supports safe concurrency.

3 Features:

- Compares concurrent behavior with serial execution.
- Used for correctness analysis.
- Important in concurrency control.

Applications: Used in multi-user database systems.

Real-Life Example: If two transactions run concurrently but produce the same result as executing T1 followed by T2, the schedule is considered serializable.

5.4 Locking Methods for Concurrency Control

Definition: Locking methods control access to database data by allowing or restricting transactions from reading or writing locked data.

3 Uses:

- Controls simultaneous data access.
- Helps prevent conflicting operations.
- Supports transaction isolation.

3 Features:

- Uses locks on data items.
- Controls read/write access.

- Applied during transaction execution.

Applications: Used in multi-user DBMSs such as banking and reservation systems.

Real-Life Example: If one transaction is updating an account record, a lock can restrict conflicting access until the required operation is completed.